

Experiment 8

Student Name: Nikhil Kumar

UID: 25MCI10036

Branch: MCA (AI/ML)

Section/Group: 25MAM-1(A)

Semester: 2-SEM

Date of Performance: 31/03/2026

Subject Name: Technical Training - 1

Subject Code: 25CAP-652

1. Aim of the Session:

To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

2. Software Requirements:

- PostgreSQL
- pgAdmin
- Windows Operating System

3. Objectives of the Session:

Apply stored procedure concepts for database operations.

4. Theory:

A Stored Procedure is a precompiled collection of SQL statements stored in the database that can be executed as a single unit. It helps in improving performance, security, and code reusability.

Features of Stored Procedures:

- Reusability
- Better Performance
- Improved Security
- Reduced Network Traffic
- Modular Programming

5. Practical / Experiment Steps:

Step 0: Creating sample tables and inserting records

Code:

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    manager_id INT,  
    department VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO Employees VALUES  
(1, 'Amit', NULL, 'Management', 120000),  
(2, 'Ravi', 1, 'Engineering', 80000),  
(3, 'Neha', 1, 'Engineering', 82000),  
(4, 'Karan', 2, 'Engineering', 60000),  
(5, 'Simran', 2, 'Engineering', 62000),  
(6, 'Pooja', 3, 'Engineering', 61000),  
(7, 'Rahul', 3, 'Engineering', 64000),  
(8, 'Arjun', 1, 'HR', 70000);
```

```
SELECT * FROM Employees;
```

Output:

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Amit	[null]	Management	120000
2	2	Ravi	1	Engineering	80000
3	3	Neha	1	Engineering	82000
4	4	Karan	2	Engineering	60000
5	5	Simran	2	Engineering	62000
6	6	Pooja	3	Engineering	61000
7	7	Rahul	3	Engineering	64000
8	8	Arjun	1	HR	70000

Step 1: Procedure to insert data.

Code:

```
CREATE OR REPLACE PROCEDURE ADD_EMPLOYEE(p_id int,p_name
varchar(50),p_manager int,p_dept varchar(50),p_salary int)
AS
$$
BEGIN
INSERT INTO Employees VALUES(p_id,p_name,p_manager,p_dept,p_salary);
END;
$$ LANGUAGE PLPGSQL;
```

```
CALL ADD_EMPLOYEE(9,'Kartik',1,'HR',68000);
```

Output:

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Amit	[null]	Management	120000
2	2	Ravi	1	Engineering	80000
3	3	Neha	1	Engineering	82000
4	4	Karan	2	Engineering	60000
5	5	Simran	2	Engineering	62000
6	6	Pooja	3	Engineering	61000
7	7	Rahul	3	Engineering	64000
8	8	Arjun	1	HR	70000
9	9	Kartik	1	HR	68000

Step 2: Procedure to update data.

Code:

```
CREATE OR REPLACE PROCEDURE UPDATE_SALARY_PROCC(IN P_EMP_ID INT,INOUT
P_SALARY NUMERIC(20,3), OUT STATUS VARCHAR(20))
AS
$$
DECLARE
CURR_SAL NUMERIC(20,3);
BEGIN

SELECT SALARY+P_SALARY INTO CURR_SAL FROM employees WHERE
EMP_ID=P_EMP_ID;
```

```
IF NOT FOUND THEN
RAISE EXCEPTION 'EMPLOYEE NOT FOUND';
END IF;

UPDATE employees
SET SALARY=CURR_SAL WHERE EMP_ID=P_EMP_ID;

P_SALARY:=CURR_SAL;
STATUS:='SUCCESS';

EXCEPTION
WHEN OTHERS THEN
IF SQLERRM LIKE '%EMPLOYEE NOT FOUND%' THEN
STATUS:='EMPLOYEE NOT FOUND';
END IF;

END;

$$ LANGUAGE PLPGSQL;
```

-- EMPLOYEE NOT FOUND

```
DO
$$
    DECLARE
    EMP_ID INT:=99;
    STATUS VARCHAR(20);
    SALARY NUMERIC(20,3):=500;
    BEGIN
    CALL UPDATE_SALARY_PROCC(EMP_ID,SALARY,STATUS);
    RAISE NOTICE 'YOUR STATUS IS %', STATUS;
    END;
$$
```

Output:

```
NOTICE:  YOUR STATUS IS EMPLOYEE NOT FOUND
DO

Query returned successfully in 78 msec.
```

-- EMPLOYEE FOUND

DO

\$\$

DECLARE

EMP_ID INT:=2;

STATUS VARCHAR(20);

SALARY NUMERIC(20,3):=500;

BEGIN

CALL UPDATE_SALARY_PROCC(EMP_ID,SALARY,STATUS);

RAISE NOTICE 'YOUR STATUS IS % AND UPDATED SALARY IS %', STATUS,

SALARY;

END;

\$\$

Output:

```
NOTICE: YOUR STATUS IS SUCCESS AND UPDATED SALARY IS 80500.000
DO
```

```
Query returned successfully in 91 msec.
```

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Amit	[null]	Management	120000
2	3	Neha	1	Engineering	82000
3	4	Karan	2	Engineering	60000
4	5	Simran	2	Engineering	62000
5	6	Pooja	3	Engineering	61000
6	7	Rahul	3	Engineering	64000
7	8	Arjun	1	HR	70000
8	9	Kartik	1	HR	68000
9	2	Ravi	1	Engineering	80500

Step 3: Procedure to delete data.

Code:

CREATE OR REPLACE PROCEDURE DELETE_EMPLOYEE(p_id int)

AS

\$\$

```
BEGIN  
DELETE FROM Employees WHERE emp_id=p_id;  
END;  
$$ LANGUAGE PLPGSQL;
```

CALL DELETE_EMPLOYEE(9);

Output:

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Amit	[null]	Management	120000
2	3	Neha	1	Engineering	82000
3	4	Karan	2	Engineering	60000
4	5	Simran	2	Engineering	62000
5	6	Pooja	3	Engineering	61000
6	7	Rahul	3	Engineering	64000
7	8	Arjun	1	HR	70000
8	2	Ravi	1	Engineering	80500

6. Learning Outcomes (What I have learned):

- Learned how to create and use stored procedures
- Understood how to perform CRUD operations using procedures
- Improved understanding of database modular programming
- Learned how to enhance security and performance
- Gained hands-on experience in PostgreSQL